

Project Topic A: Parallel Machine Scheduling

Problem Solving and Search in AI 2026

This is one of the two problems you can choose for assignments. You can choose the same problem for all assignments or work on a different problem for each assignment.

1 The Parallel Machine Scheduling Problem

Modern industrial factories use sophisticated parallel machine environments to schedule a large number of jobs to manufacture products.

Thus, we need to find schedules that fulfill all technical constraints (e.g. regarding the availability of resources), while the production costs should be minimized.

1.1 Problem Description

The input to our problem is a set of jobs and a set of machines, and our goal is to find a feasible and cost-optimized schedule for all jobs on the given machines. In addition, several constraints on resource usage and job precedences must be met.

Figure 1 illustrates a candidate schedule for 6 jobs ($J1$ - $J6$) on two machines $M1$ and $M2$. The horizontal axis represents time (in this example, from 0 to 10), and the vertical axis represents the machines and the resource information.

Thus, we see that jobs $J1$, $J3$, $J6$ and $J4$ in this example are assigned to machine $M1$, whereas jobs $J2$ and $J5$ are assigned to machine $M2$. Furthermore, the horizontal length of the colored bars in Figure 1 represents the processing time of each job (e.g., $J1$ starts at time 0 and ends at time 2, and $J2$ starts at time 1 and ends at time 2).

At the top of Figure 1, information about the available resource capacities over time is displayed. *Resource 1* has 4 units of capacity available from time 0 to 5 and 2 units of capacity from time 6 to 9. No capacity is available for *Resource 1* from time 5 to 6 and time 9 to 10 (capacity is 0). *Resource 2* has 2 units of capacity available from time 3 to time 9, and no capacity is available for *Resource 2* from time 0 to 3 and 9 to 10. Different jobs may have different resource capacity requirements, and we need to ensure that resource capacities are available for all running jobs at every time point in the schedule. E.g., jobs $J3$ and $J5$ are running in parallel during time 4 to 5, and both

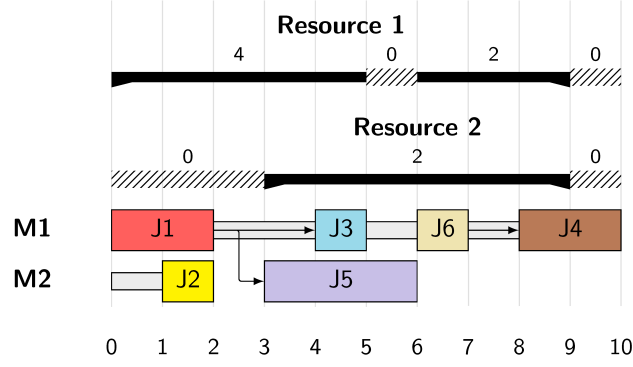


Figure 1: An illustration of a solution schedule for jobs $J1$ - $J6$ on machines $M1$ and $M2$.

need 1 capacity of *Resource 2* in our example. This is valid as 2 resource capacity units are available during this time.

The arrows shown in Figure 1 further visualize the precedence requirements between pairs of jobs. I.e., the arrows from the end of the job $J1$ to the start of $J3$ and $J5$ indicate that $J1$ must be finished before $J3$ and $J5$ can start. In addition, the job $J6$ must be completed before $J4$ can start.

Finally, the gray bars between some jobs indicate that a setup time is required between consecutively scheduled jobs. In these cases, additional time (represented by the length of the gray horizontal bar) is needed to prepare the machine before the next job can start.

The following list summarizes all requirements that must be satisfied by a feasible schedule:

- *Machine Eligibility*: Some jobs can only be scheduled on a subset of the given machines, which are eligible. The eligible machines are defined for each job individually.
- *Non-overlapping jobs*: No jobs can run in parallel on a single machine. Thus, at any time, no two jobs can overlap on a single machine.
- *Job precedences*: Each job defines a set of precedence jobs that must be completed before the job can start (the start time can be equal to the end time of a precedence job).
- *Setup Times*: A job defines the setup times that are needed before it can start after the predecessor job has finished (this time can vary depending on the predecessor job). The start time of the job must therefore be greater than or equal to the end time of the predecessor job plus the associated setup time. Each job further defines the initial setup time required if it is scheduled as the first job on a machine.
- *Jobs resource requirements must be met without overflowing the available resource capacity*: Each job can require different resource capacities. These resource ca-

capacities must be available throughout the job's processing time. Jobs running in parallel on different machines may simultaneously consume capacity from the same resource, as long as the total available capacity is sufficient. After a job is completed, all required resource capacities are released again.

Furthermore, the following two objectives measure the cost of a schedule:

- *Tardiness*: Each job defines the due time for completion. This is not a hard constraint, and jobs may be scheduled late. However, tardiness creates costs that should be minimized. Let the set of jobs be J , the end time for each job be end_j , and the due time be due_j ($\forall j \in J$). The total tardiness is then defined as follows:

$$tardiness = \sum_{j \in J} \max\{0, end_j - due_j\} \quad (1)$$

- *Makespan*: The makespan of a schedule is equivalent to the completion time of the last job in the schedule and is defined as follows:

$$makespan = \max_{j \in J} \{end_j\} \quad (2)$$

The aim is to find a schedule that optimizes tardiness and makespan, and thus the minimization objective is defined as the sum of tardiness and makespan:

$$minimize \sum_{j \in J} \max\{0, end_j - due_j\} + \max_{j \in J} \{end_j\} \quad (3)$$

2 Benchmark Instances

To evaluate your approaches, you should use the benchmark instances available in TUWEL. Include an analysis of the behavior of your methods for these instances in your evaluation and provide your solutions in a folder named **results** in your submission ZIP file using `PSSAI_PMS_jX_mX_rX_X.solution.json` as file names (replace **X** by the parameters of each instance).

In the Topic A ZIP archive available in TUWEL, you will find a set of 20 problem instances, including smaller instances and large-scale industrial problems. In addition, the archive includes a toy instance, its corresponding solution, and a Python script for validation. Each instance is formatted as a JSON text file. For example, part of the contents of the toy instance, similar to the example shown in Figure 1, are as follows:

```
{
  "Machines": [
    {
      "Id": 1,
      "Name": "M1"
    },
  ],
}
```

```

{
  "Id": 2,
  "Name": "M2"
}
],
"Jobs": [
  {
    "Id": 1,
    "Name": "J1",
    "ProcessingTime": 2,
    "DueTime": 3,
    "EligibleMachineIds": [
      1,
      2
    ],
    "RequiredResources": [
      {
        "ResourceId": 1,
        "Capacity": 1
      }
    ],
    "PrecedenceJobIds": [
    ],
    "InitialSetupTime": 0,
    "JobSetupTimes": [
      0,
      0,
      3,
      0,
      5,
      6
    ]
  }
],
{
  "Id": 2,
  "Name": "J2",
  "ProcessingTime": 1,
  "DueTime": 3,
  "EligibleMachineIds": [
    1,
    2
  ],
  "RequiredResources": [
    {

```

```

        "ResourceId": 1,
        "Capacity": 3
    }
],
"PrecedenceJobIds": [
],
"InitialSetupTime": 1,
"JobSetupTimes": [
    1,
    0,
    1,
    1,
    1,
    1
]
},
[...]
{
    "Id": 6,
    "Name": "J6",
    "ProcessingTime": 1,
    "DueTime": 5,
    "EligibleMachineIds": [
        1,
        2
    ],
    "RequiredResources": [
    ],
    "PrecedenceJobIds": [
    ],
    "InitialSetupTime": 2,
    "JobSetupTimes": [
        2,
        2,
        1,
        3,
        1,
        0
    ]
}
],
"Resources": [
    {
        "Id": 1,

```

```

    "Name": "Resource 1",
    "AvailabilityPeriods": [
      {
        "Start": 0,
        "End": 5,
        "Capacity": 4
      },
      {
        "Start": 5,
        "End": 6,
        "Capacity": 0
      },
      {
        "Start": 6,
        "End": 9,
        "Capacity": 2
      }
    ]
  },
  {
    "Id": 2,
    "Name": "Resource 2",
    "AvailabilityPeriods": [
      {
        "Start": 0,
        "End": 3,
        "Capacity": 0
      },
      {
        "Start": 3,
        "End": 9,
        "Capacity": 2
      }
    ]
  }
]
}

```

The following instance attributes are given in an instance file:

- *Machines*: A list containing all machine IDs and names.
- *Jobs*: A list containing all the job information.
 - *Id*: The ID of the job.

- *Name*: The name of the job.
 - *ProcessingTime*: The job’s processing time.
 - *DueTime*: The job’s due time (necessary to calculate the tardiness objective).
 - *EligibleMachineIds*: The set of machine IDs that are eligible for this job. The job can only be scheduled to one of these machines.
 - *RequiredResources*: The resource capacities that are required throughout the job’s processing time.
 - *PrecedenceJobIds*: The set of job IDs that must be completed before this job can start.
 - *InitialSetupTime*: The required setup time if this job is scheduled first on any machine.
 - *JobSetupTime*: The required setup time for each possible predecessor job ID. The positions of the list match the job IDs (i.e., in the toy example, job J1 requires a setup time of 0 if its predecessor is J2, a setup time of 3 if its predecessor is J3, etc.).
- *Resources*: A list containing information on the available resources.
 - *Id*: The ID of the resource.
 - *Name*: The name of the resource.
 - *AvailabilityPeriods*: List of all availability periods for this resource. The capacities can vary from period to period (as shown in Figure 1). Note that periods are defined seamlessly from time 0 to the end time of the last period (e.g., 9 in the toy instance). After the last defined period ends, we assume that the capacity of this resource remains 0 indefinitely.

2.1 Solution Format

To use the solution validator Python script, the solutions should be saved in JSON format. For example, the contents of a possible solution to the toy instance are as follows:

```
{
  "Jobs": [
    {
      "JobId": 1,
      "StartTime": 0,
      "MachineId": 1
    },
    {
      "JobId": 2,
      "StartTime": 1,
      "MachineId": 2
    }
  ]
}
```

```

    },
    {
        "JobId": 3,
        "StartTime": 4,
        "MachineId": 1
    },
    {
        "JobId": 4,
        "StartTime": 7,
        "MachineId": 1
    },
    {
        "JobId": 5,
        "StartTime": 6,
        "MachineId": 2
    },
    {
        "JobId": 6,
        "StartTime": 4,
        "MachineId": 2
    }
]
}

```

The following solution attributes are given in a solution file:

- *Jobs*: A list containing information on the scheduled jobs.
 - *JobId*: The ID of the scheduled job.
 - *StartTime*: The starting time of the job.
 - *MachineId*: The machine ID the job is assigned to.

3 Hints & Additional Remarks

- Use the Python solution validator script to check if your solutions for each instance are feasible and do not violate any constraints. If you think there is a bug or a mistake in the solution validator, please contact us.
- With this problem, you have the opportunity to challenge us and beat our best results. The best results for each instance will be uploaded to TUWEL later. However, it is not necessary to beat our results for a good grade. An efficient and well-evaluated approach is enough.